

Chapter 54 — Design Data Structures

0. What You Will Learn in This Chapter

Design Data Structure problems ask you to implement a class with specific APIs and time complexity requirements.

These problems are common in interviews because they test whether you can combine multiple data structures to support operations efficiently.

Typical requirements:

- insert in $O(1)$
- delete in $O(1)$
- getRandom in $O(1)$
- getMin in $O(1)$
- getMax in $O(1)$
- getMedian dynamically
- cache eviction
- frequency tracking
- timestamp ordering
- custom iterators
- file-system-like paths
- simulation with efficient state

By the end of this chapter, you should understand:

- Design problem mindset
- How to choose internal data structures
- How to combine hash maps, linked lists, heaps, sets, stacks, and queues
- How to implement LRU Cache
- How to implement LFU Cache
- How to implement Min Stack
- How to implement Max Stack
- How to implement RandomizedSet
- How to implement AllOne Data Structure
- How to implement Median Finder
- How to design Twitter
- How to simulate Snake Game
- How to design a simple File System
- Common design interview traps

Core problems covered in this chapter:

1. Min Stack
2. Max Stack
3. LRU Cache
4. LFU Cache
5. Insert Delete GetRandom $O(1)$
6. Insert Delete GetRandom $O(1)$ - Duplicates Allowed
7. All $O(1)$ Data Structure
8. Find Median from Data Stream
9. Design Twitter

10. Design Snake Game
 11. Design File System
 12. Design In-Memory File System
 13. Moving Average from Data Stream
 14. Recent Counter
 15. Hit Counter
 16. Time Based Key-Value Store
 17. Snapshot Array
 18. Peeking Iterator
 19. Design Browser History
-

1. What Makes Design Problems Different?

Most algorithm problems ask for one function.
Design problems ask you to implement a class:

```
class LRUCache {  
public:  
    LRUCache(int capacity);  
    int get(int key);  
    void put(int key, int value);  
};
```

You must maintain internal state across calls.
This means you need to think about:

- data representation
 - operation complexity
 - updates after every API call
 - edge cases across sequences of operations
 - stale data
 - duplicate entries
 - eviction rules
 - memory consistency
-

2. The Main Design Mindset

For every API operation, ask:

20. What information do I need to answer this operation?
21. What information changes after this operation?
22. Which operation needs to be fastest?
23. Is one data structure enough?
24. If not, how do multiple data structures stay synchronized?

Most design problems are solved by combining data structures.
Examples:

fast lookup by key	hash map
maintain order	linked list / set
remove arbitrary node	linked list with pointer
get min/max	heap / ordered set / monotonic stack
get random	vector
delete random element $O(1)$	vector + hashmap
get median	two heaps
frequency buckets	hashmap + linked lists
time search	hashmap + sorted vector
path hierarchy	trie / hash tree

3. Data Structure Composition

Design problems often require two or more structures.

3.1 Hash Map + Doubly Linked List

Used for:

LRU Cache

Hash map gives $O(1)$ lookup.

Doubly linked list gives $O(1)$ move-to-front and remove-tail.

3.2 Hash Map + Vector

Used for:

RandomizedSet

Hash map gives $O(1)$ index lookup.

Vector gives $O(1)$ random access.

3.3 Two Heaps

Used for:

Median Finder

Max heap stores lower half.

Min heap stores upper half.

3.4 Hash Map + Frequency Buckets

Used for:

LFU Cache

AllOne Data Structure

Need track counts and update buckets efficiently.

3.5 Hash Map + Sorted Vector

Used for:

Time Based Key-Value Store

Snapshot Array

Need binary search over timestamps or versions.

4. Pattern Recognition Checklist

Ask:

25. Do I need $O(1)$ lookup by key?
26. Do I need to evict least recently used item?
27. Do I need to evict least frequently used item?
28. Do I need `getMin` or `getMax` in $O(1)$?
29. Do I need `getRandom` in $O(1)$?
30. Do I need median dynamically?
31. Do I need timestamps or versions?
32. Do I need path-based lookup?
33. Do I need maintain order of events?
34. Do I need remove arbitrary elements quickly?
35. Can stale heap entries appear?
36. How will I keep structures synchronized?

5. Common Internal Building Blocks

5.1 Doubly Linked List Node

```
struct Node {  
    int key;  
    int value;  
    Node* prev;  
    Node* next;  
  
    Node(int k, int v) {  
        key = k;  
        value = v;  
        prev = nullptr;  
        next = nullptr;  
    }  
};
```

Useful when you must remove or move nodes in $O(1)$.

5.2 Dummy Head and Tail

Dummy nodes simplify edge cases.

head <-> real nodes <-> tail

No need to handle empty list separately for insert/remove.

```
void remove(Node* node) {
    node->prev->next = node->next;
    node->next->prev = node->prev;
}

void insertAfterHead(Node* node) {
    node->next = head->next;
    node->prev = head;

    head->next->prev = node;
    head->next = node;
}
```

5.3 Two Heaps

```
priority_queue<int> low; // max heap
priority_queue<int, vector<int>, greater<int>> high; // min heap
```

Invariant:

```
low.size() == high.size()
or
low.size() == high.size() + 1
```

5.4 Vector + Index Map

```
vector<int> values;
unordered_map<int, int> index;
```

Delete by swapping with last:

```
int idx = index[val];
int last = values.back();

values[idx] = last;
index[last] = idx;

values.pop_back();
index.erase(val);
```

6. Full Problem Walkthrough 1 — LeetCode 155: Min Stack

6.1 Problem Summary

Design stack supporting:

push
pop
top
getMin

All in $O(1)$.

6.2 Key Idea

Maintain another stack of current minimum values.

At every push, store:

value
current minimum so far

6.3 C++ Code

```
class MinStack {
private:
    stack<pair<int, int>> st;

public:
    MinStack() {
    }

    void push(int val) {
        int currentMin = val;

        if (!st.empty()) {
            currentMin = min(currentMin, st.top().second);
        }

        st.push({val, currentMin});
    }

    void pop() {
        st.pop();
    }

    int top() {
        return st.top().first;
    }

    int getMin() {
        return st.top().second;
    }
};
```

```
}  
};
```

6.4 Why This Works

Each stack entry remembers the minimum at the moment it was pushed.
When popped, previous minimum is automatically restored.

7. Full Problem Walkthrough 2 — Max Stack

7.1 Problem Summary

Design stack supporting:

```
push  
pop  
top  
peekMax  
popMax
```

popMax removes the top-most maximum if duplicates exist.

7.2 Approach

Use:

```
list<int> dll  
map<int, vector<list<int>::iterator>> positions
```

Why?

- list supports $O(1)$ erase by iterator.
- map gives largest value using rbegin.
- vector stores iterators for each value in stack order.

7.3 C++ Code

```
class MaxStack {  
private:  
    list<int> dll;  
    map<int, vector<list<int>::iterator>> pos;  
  
public:  
    MaxStack() {  
    }  
  
    void push(int x) {  
        dll.push_back(x);  
        auto it = prev(dll.end());  
        pos[x].push_back(it);  
    }  
}
```

```

int pop() {
    int x = dll.back();

    dll.pop_back();

    pos[x].pop_back();

    if (pos[x].empty()) {
        pos.erase(x);
    }

    return x;
}

int top() {
    return dll.back();
}

int peekMax() {
    return pos.rbegin()->first;
}

int popMax() {
    auto mapIt = prev(pos.end());

    int maxVal = mapIt->first;

    auto listIt = mapIt->second.back();

    dll.erase(listIt);

    mapIt->second.pop_back();

    if (mapIt->second.empty()) {
        pos.erase(mapIt);
    }

    return maxVal;
}
};

```

7.4 Complexity

push: $O(\log n)$
pop: $O(\log n)$
top: $O(1)$
peekMax: $O(1)$
popMax: $O(\log n)$

8. Full Problem Walkthrough 3 — LeetCode 146: LRU Cache

8.1 Problem Summary

Design cache with fixed capacity.

Operations:

get(key)
put(key, value)

Both $O(1)$.

Evict least recently used key when capacity exceeded.

8.2 Key Idea

Need:

- $O(1)$ lookup by key
- $O(1)$ update recency
- $O(1)$ remove least recent

Use:

unordered_map<int, Node*>
doubly linked list

Most recent near head.

Least recent near tail.

8.3 C++ Code

```
class LRUCache {
private:
    struct Node {
        int key;
        int value;
        Node* prev;
        Node* next;

        Node(int k, int v) {
            key = k;
            value = v;
            prev = nullptr;
            next = nullptr;
        }
    };
};
```

```

    }
};

int capacity;
unordered_map<int, Node*> mp;
Node* head;
Node* tail;

void remove(Node* node) {
    node->prev->next = node->next;
    node->next->prev = node->prev;
}

void insertAfterHead(Node* node) {
    node->next = head->next;
    node->prev = head;

    head->next->prev = node;
    head->next = node;
}

void moveToFront(Node* node) {
    remove(node);
    insertAfterHead(node);
}

public:
    LRUCache(int capacity) {
        this->capacity = capacity;

        head = new Node(-1, -1);
        tail = new Node(-1, -1);

        head->next = tail;
        tail->prev = head;
    }

    int get(int key) {
        if (!mp.count(key)) {
            return -1;
        }
    }

```

```

    Node* node = mp[key];

    moveToFront(node);

    return node->value;
}

void put(int key, int value) {
    if (mp.count(key)) {
        Node* node = mp[key];
        node->value = value;
        moveToFront(node);
        return;
    }

    Node* node = new Node(key, value);
    mp[key] = node;
    insertAfterHead(node);

    if (mp.size() > capacity) {
        Node* victim = tail->prev;

        remove(victim);
        mp.erase(victim->key);

        delete victim;
    }
}
};

```

8.4 Important Details

On get, update recency.

On updating existing key with put, also update recency.

Evict from tail.

Dummy nodes prevent many edge-case bugs.

9. Full Problem Walkthrough 4 — LeetCode 460: LFU Cache

9.1 Problem Summary

Design cache with:

get
put

Evict least frequently used key.

If tie, evict least recently used among those with same frequency.

9.2 Required Structures

Need:

key -> value
key -> frequency
frequency -> list of keys ordered by recency
key -> iterator in its frequency list
minFreq

When a key is accessed:

- remove from old frequency list
- increase frequency
- insert into new frequency list front
- update minFreq if old list becomes empty

9.3 C++ Code

```
class LFUCache {
private:
    int capacity;
    int minFreq;

    unordered_map<int, int> value;
    unordered_map<int, int> freq;
    unordered_map<int, list<int>> freqList;
    unordered_map<int, list<int>::iterator> iter;

    void touch(int key) {
        int f = freq[key];

        freqList[f].erase(iter[key]);

        if (freqList[f].empty()) {
            freqList.erase(f);

            if (minFreq == f) {
                minFreq++;
            }
        }

        freq[key]++;
    }
};
```

```

    int newFreq = f + 1;

    freqList[newFreq].push_front(key);
    iter[key] = freqList[newFreq].begin();
}

public:
    LFUCache(int capacity) {
        this->capacity = capacity;
        minFreq = 0;
    }

    int get(int key) {
        if (!value.count(key)) {
            return -1;
        }

        touch(key);

        return value[key];
    }

    void put(int key, int val) {
        if (capacity == 0) {
            return;
        }

        if (value.count(key)) {
            value[key] = val;
            touch(key);
            return;
        }

        if (value.size() == capacity) {
            int victim = freqList[minFreq].back();

            freqList[minFreq].pop_back();

            if (freqList[minFreq].empty()) {
                freqList.erase(minFreq);
            }
        }
    }

```

```

        value.erase(victim);
        freq.erase(victim);
        iter.erase(victim);
    }

    value[key] = val;
    freq[key] = 1;

    freqList[1].push_front(key);
    iter[key] = freqList[1].begin();

    minFreq = 1;
}
};

```

9.4 Complexity

get: $O(1)$
 put: $O(1)$

Average $O(1)$, assuming unordered maps.

10. Full Problem Walkthrough 5 — LeetCode 380: RandomizedSet

10.1 Problem Summary

Support:

```

insert(val)
remove(val)
getRandom()

```

All average $O(1)$.

10.2 Key Idea

Use:

```

vector<int> values
unordered_map<int, int> index

```

Vector gives random access.

Map gives index lookup.

Remove by swap-with-last.

10.3 C++ Code

```

class RandomizedSet {
private:

```

```

vector<int> values;
unordered_map<int, int> index;

public:
    RandomizedSet() {
    }

    bool insert(int val) {
        if (index.count(val)) {
            return false;
        }

        index[val] = values.size();
        values.push_back(val);

        return true;
    }

    bool remove(int val) {
        if (!index.count(val)) {
            return false;
        }

        int idx = index[val];
        int lastVal = values.back();

        values[idx] = lastVal;
        index[lastVal] = idx;

        values.pop_back();
        index.erase(val);

        return true;
    }

    int getRandom() {
        int idx = rand() % values.size();

        return values[idx];
    }
};

```

10.4 Edge Case

Removing the last element still works.
If `val == lastVal`, we update index then erase it.

11. Full Problem Walkthrough 6 — RandomizedCollection With Duplicates

11.1 Problem Summary

Like `RandomizedSet`, but duplicates are allowed.

11.2 Key Difference

Need store multiple indices per value:

```
unordered_map<int, unordered_set<int>> indices;
```

11.3 C++ Code

```
class RandomizedCollection {
private:
    vector<int> values;
    unordered_map<int, unordered_set<int>> indices;

public:
    RandomizedCollection() {}

    bool insert(int val) {
        bool isNew = indices[val].empty();

        indices[val].insert(values.size());
        values.push_back(val);

        return isNew;
    }

    bool remove(int val) {
        if (indices[val].empty()) {
            return false;
        }

        int removeldx = *indices[val].begin();
        indices[val].erase(removeldx);
```

```

int lastVal = values.back();
int lastIdx = values.size() - 1;

values[removeldx] = lastVal;

if (removeldx != lastIdx) {
    indices[lastVal].erase(lastIdx);
    indices[lastVal].insert(removeldx);
}

values.pop_back();

return true;
}

int getRandom() {
    return values[rand() % values.size()];
}
};

```

12. Full Problem Walkthrough 7 — LeetCode 432: All O(1) Data Structure

12.1 Problem Summary

Design data structure supporting:

```

inc(key)
dec(key)
getMaxKey()
getMinKey()

```

All O(1).
 Each key has a count.
 Need return any key with max count or min count.

12.2 Key Idea

Use bucketed doubly linked list.
 Each bucket contains:

```

count
set of keys with that count

```

Buckets are ordered by count.
 Need:

key -> bucket iterator

Then:

- inc moves key to count + 1 bucket
- dec moves key to count - 1 bucket
- remove empty buckets
- min is front bucket
- max is back bucket

12.3 C++ Code

```
class AllOne {
private:
    struct Bucket {
        int count;
        unordered_set<string> keys;

        Bucket(int c) {
            count = c;
        }
    };

    list<Bucket> buckets;
    unordered_map<string, list<Bucket>::iterator> where;

public:
    AllOne() {
    }

    void inc(string key) {
        if (!where.count(key)) {
            if (buckets.empty() || buckets.front().count != 1) {
                buckets.push_front(Bucket(1));
            }

            buckets.front().keys.insert(key);
            where[key] = buckets.begin();
            return;
        }

        auto it = where[key];
        int newCount = it->count + 1;

        auto nextIt = next(it);
```

```

    if (nextIt == buckets.end() || nextIt->count != newCount) {
        nextIt = buckets.insert(nextIt, Bucket(newCount));
    }

    nextIt->keys.insert(key);
    where[key] = nextIt;

    it->keys.erase(key);

    if (it->keys.empty()) {
        buckets.erase(it);
    }
}

void dec(string key) {
    auto it = where[key];

    if (it->count == 1) {
        it->keys.erase(key);
        where.erase(key);

        if (it->keys.empty()) {
            buckets.erase(it);
        }

        return;
    }

    int newCount = it->count - 1;

    auto prevIt = it;

    if (it == buckets.begin()) {
        prevIt = buckets.end();
    } else {
        prevIt = prev(it);
    }

    if (it == buckets.begin() || prevIt->count != newCount) {
        prevIt = buckets.insert(it, Bucket(newCount));
    }
}

```

```

prevIt->keys.insert(key);
where[key] = prevIt;

it->keys.erase(key);

if (it->keys.empty()) {
    buckets.erase(it);
}
}

string getMaxKey() {
    if (buckets.empty()) {
        return "";
    }

    return *buckets.back().keys.begin();
}

string getMinKey() {
    if (buckets.empty()) {
        return "";
    }

    return *buckets.front().keys.begin();
}
};

```

12.4 Complexity

All operations:

$O(1)$

Average $O(1)$, assuming hash operations are $O(1)$.

13. Full Problem Walkthrough 8 — LeetCode 295: Find Median from Data Stream

13.1 Problem Summary

Support:

```

addNum(num)
findMedian()

```

13.2 Key Idea

Use two heaps:

low = max heap for smaller half

high = min heap for larger half

Keep sizes balanced.

Invariant:

low.size() == high.size()

or

low.size() == high.size() + 1

13.3 C++ Code

```
class MedianFinder {
private:
    priority_queue<int> low;
    priority_queue<int, vector<int>, greater<int>> high;

public:
    MedianFinder() {
    }

    void addNum(int num) {
        if (low.empty() || num <= low.top()) {
            low.push(num);
        } else {
            high.push(num);
        }

        if (low.size() > high.size() + 1) {
            high.push(low.top());
            low.pop();
        } else if (high.size() > low.size()) {
            low.push(high.top());
            high.pop();
        }
    }

    double findMedian() {
        if (low.size() > high.size()) {
            return low.top();
        }

        return ((double)low.top() + high.top()) / 2.0;
    }
};
```

```
}  
};
```

13.4 Complexity

addNum: $O(\log n)$
findMedian: $O(1)$

14. Full Problem Walkthrough 9 — LeetCode 355: Design Twitter

14.1 Problem Summary

Implement simplified Twitter:

```
postTweet(userId, tweetId)  
getNewsFeed(userId)  
follow(followerId, followeeId)  
unfollow(followerId, followeeId)
```

News feed returns 10 most recent tweet IDs by user and users they follow.

14.2 Key Structures

user -> set of followees
user -> vector of tweets
global timestamp

For feed:

- collect recent tweets from user and followees
- use max heap by timestamp

14.3 C++ Code

```
class Twitter {  
private:  
    int timeStamp;  
  
    unordered_map<int, unordered_set<int>> follows;  
    unordered_map<int, vector<pair<int, int>>> tweets;  
    // user -> vector of {time, tweetId}  
  
public:  
    Twitter() {  
        timeStamp = 0;  
    }  
  
    void postTweet(int userId, int tweetId) {  
        tweets[userId].push_back({timeStamp++, tweetId});  
    }  
};
```

```

}

vector<int> getNewsFeed(int userId) {
    vector<int> result;

    priority_queue<pair<int, int>> pq;

    follows[userId].insert(userId);

    for (int followee : follows[userId]) {
        auto& arr = tweets[followee];

        int count = 0;

        for (int i = arr.size() - 1; i >= 0 && count < 10; i--, count++) {
            pq.push(arr[i]);
        }
    }

    while (!pq.empty() && result.size() < 10) {
        result.push_back(pq.top().second);
        pq.pop();
    }

    return result;
}

void follow(int followerId, int followeeId) {
    follows[followerId].insert(followeeId);
}

void unfollow(int followerId, int followeeId) {
    if (followerId == followeeId) {
        return;
    }

    follows[followerId].erase(followeeId);
}
};

```

14.4 Note

This simple approach is accepted for typical constraints.
More scalable design uses k-way merge with tweet pointers.

15. Full Problem Walkthrough 10 — LeetCode 353: Design Snake Game

15.1 Problem Summary

Snake moves on grid.

Need support:

```
move(direction)
```

Return score or -1 if game over.

Snake grows when eating food.

Collision with wall or itself ends game.

15.2 Key Structures

Use:

```
deque<pair<int,int>> body
unordered_set<int> occupied
food index
```

Encode cell:

```
id = row * width + col
```

Important:

Before checking self-collision, remove tail if snake is not growing.

Why?

Moving into the old tail position is allowed if tail moves away.

15.3 C++ Code

```
class SnakeGame {
private:
    int width;
    int height;
    vector<vector<int>> food;
    int foodIndex;

    deque<pair<int, int>> body;
    unordered_set<int> occupied;

    int encode(int r, int c) {
        return r * width + c;
    }

public:
    SnakeGame(int width, int height, vector<vector<int>>& food) {
        this->width = width;
```

```

this->height = height;
this->food = food;
foodIndex = 0;

body.push_front({0, 0});
occupied.insert(encode(0, 0));
}

int move(string direction) {
    auto [r, c] = body.front();

    if (direction == "U") {
        r--;
    } else if (direction == "D") {
        r++;
    } else if (direction == "L") {
        c--;
    } else {
        c++;
    }

    if (r < 0 || r >= height || c < 0 || c >= width) {
        return -1;
    }

    bool grow = false;

    if (foodIndex < food.size() &&
        food[foodIndex][0] == r &&
        food[foodIndex][1] == c) {
        grow = true;
        foodIndex++;
    }

    if (!grow) {
        auto tail = body.back();
        occupied.erase(encode(tail.first, tail.second));
        body.pop_back();
    }

    int newId = encode(r, c);

```

```

        if (occupied.count(newId)) {
            return -1;
        }

        body.push_front({r, c});
        occupied.insert(newId);

        return body.size() - 1;
    }
};

```

16. Full Problem Walkthrough 11 — Time Based Key-Value Store

16.1 Problem Summary

Implement:

```

set(key, value, timestamp)
get(key, timestamp)

```

Return value with largest timestamp $\leq$ given timestamp.

16.2 Key Idea

For each key, store sorted vector of timestamp-value pairs.

Timestamps are usually given increasing for each key.

Use binary search.

16.3 C++ Code

```

class TimeMap {
private:
    unordered_map<string, vector<pair<int, string>>> mp;

public:
    TimeMap() {
    }

    void set(string key, string value, int timestamp) {
        mp[key].push_back({timestamp, value});
    }

    string get(string key, int timestamp) {
        if (!mp.count(key)) {
            return "";
        }
    }
}

```

```

auto& arr = mp[key];

int left = 0;
int right = arr.size() - 1;

string answer = "";

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (arr[mid].first <= timestamp) {
        answer = arr[mid].second;
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return answer;
}
};

```

16.4 Complexity

set: $O(1)$
get: $O(\log n)$

where n is number of values for the key.

17. Full Problem Walkthrough 12 — Snapshot Array

17.1 Problem Summary

Support:

```

set(index, val)
snap()
get(index, snap_id)

```

Each snap returns an ID.

Need get value at index at specific snapshot.

17.2 Key Idea

For each index, store changes:

```
vector<pair<snapId, value>>
```

To get value at snap_id, binary search last entry with snapId <= requested.

17.3 C++ Code

```
class SnapshotArray {
private:
    int currentSnap;
    vector<vector<pair<int, int>>> history;

public:
    SnapshotArray(int length) {
        currentSnap = 0;
        history.resize(length);

        for (int i = 0; i < length; i++) {
            history[i].push_back({0, 0});
        }
    }

    void set(int index, int val) {
        if (history[index].back().first == currentSnap) {
            history[index].back().second = val;
        } else {
            history[index].push_back({currentSnap, val});
        }
    }

    int snap() {
        currentSnap++;
        return currentSnap - 1;
    }

    int get(int index, int snap_id) {
        auto& arr = history[index];

        int left = 0;
        int right = arr.size() - 1;

        int answer = 0;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (arr[mid].first <= snap_id) {
```

```
        answer = arr[mid].second;
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return answer;
}
};
```

18. Full Problem Walkthrough 13 — Moving Average from Data Stream

18.1 Problem Summary

Given window size.

Each call:

```
next(val)
```

returns moving average of last size values.

18.2 C++ Code

```
class MovingAverage {
private:
    queue<int> q;
    int size;
    long long sum;

public:
    MovingAverage(int size) {
        this->size = size;
        sum = 0;
    }

    double next(int val) {
        q.push(val);
        sum += val;

        if (q.size() > size) {
            sum -= q.front();
            q.pop();
        }
    }
};
```

```
        return (double)sum / q.size();
    }
};
```

19. Full Problem Walkthrough 14 — Recent Counter

19.1 Problem Summary

Each call:

ping(t)

returns number of calls in range:

[t - 3000, t]

19.2 C++ Code

```
class RecentCounter {
private:
    queue<int> q;

public:
    RecentCounter() {
    }

    int ping(int t) {
        q.push(t);

        while (!q.empty() && q.front() < t - 3000) {
            q.pop();
        }

        return q.size();
    }
};
```

20. Full Problem Walkthrough 15 — Hit Counter

20.1 Problem Summary

Design hit counter.

Support:

```
hit(timestamp)
getHits(timestamp)
```

Return hits in past 5 minutes.

5 minutes = 300 seconds.

20.2 Simple Queue Code

```
class HitCounter {
private:
    queue<int> q;

public:
    HitCounter() {
    }

    void hit(int timestamp) {
        q.push(timestamp);
    }

    int getHits(int timestamp) {
        while (!q.empty() && q.front() <= timestamp - 300) {
            q.pop();
        }

        return q.size();
    }
};
```

20.3 Optimized Bucket Version

If many hits occur at same timestamp, use fixed arrays of size 300.

```
class HitCounter {
private:
    vector<int> times;
    vector<int> hits;

public:
    HitCounter() {
        times.assign(300, 0);
        hits.assign(300, 0);
    }

    void hit(int timestamp) {
        int idx = timestamp % 300;
```

```

        if (times[idx] != timestamp) {
            times[idx] = timestamp;
            hits[idx] = 1;
        } else {
            hits[idx]++;
        }
    }

    int getHits(int timestamp) {
        int total = 0;

        for (int i = 0; i < 300; i++) {
            if (timestamp - times[i] < 300) {
                total += hits[i];
            }
        }

        return total;
    }
};

```

21. Full Problem Walkthrough 16 — Design Browser History

21.1 Problem Summary

Support:

```

visit(url)
back(steps)
forward(steps)

```

21.2 Key Idea

Use vector and current index.

When visiting new page after going back, discard forward history.

21.3 C++ Code

```

class BrowserHistory {
private:
    vector<string> history;
    int current;

public:

```

```
BrowserHistory(string homepage) {  
    history.push_back(homepage);  
    current = 0;  
}  
  
void visit(string url) {  
    history.resize(current + 1);  
  
    history.push_back(url);  
    current++;  
}  
  
string back(int steps) {  
    current = max(0, current - steps);  
  
    return history[current];  
}  
  
string forward(int steps) {  
    current = min((int)history.size() - 1, current + steps);  
  
    return history[current];  
}  
};
```

22. File System Design Preview

22.1 Basic File System

Path example:

```
/a/b/c
```

Need support:

```
createPath(path, value)  
get(path)
```

Use a trie-like tree.

Each node:

```
unordered_map<string, Node*> children;  
int value;  
bool hasValue;
```

22.2 Simple Code

```
class FileSystem {
private:
    struct Node {
        unordered_map<string, Node*> children;
        int value;
        bool hasValue;

        Node() {
            value = -1;
            hasValue = false;
        }
    };

    Node* root;

    vector<string> split(string path) {
        vector<string> parts;
        string current;

        for (char c : path) {
            if (c == '/') {
                if (!current.empty()) {
                    parts.push_back(current);
                    current.clear();
                }
            } else {
                current.push_back(c);
            }
        }

        if (!current.empty()) {
            parts.push_back(current);
        }

        return parts;
    }

public:
    FileSystem() {
        root = new Node();
    }
}
```

```

bool createPath(string path, int value) {
    vector<string> parts = split(path);

    Node* node = root;

    for (int i = 0; i < parts.size(); i++) {
        string part = parts[i];

        if (i == parts.size() - 1) {
            if (node->children.count(part)) {
                return false;
            }

            node->children[part] = new Node();
            node->children[part]->value = value;
            node->children[part]->hasValue = true;

            return true;
        }

        if (!node->children.count(part)) {
            return false;
        }

        node = node->children[part];
    }

    return false;
}

int get(string path) {
    vector<string> parts = split(path);

    Node* node = root;

    for (string part : parts) {
        if (!node->children.count(part)) {
            return -1;
        }

        node = node->children[part];
    }
}

```

```
        return node->hasValue ? node->value : -1;
    }
};
```

23. Design In-Memory File System Preview

A more complete file system may need:

```
ls(path)
mkdir(path)
addContentToFile(filePath, content)
readContentFromFile(filePath)
```

Node fields:

```
map<string, Node*> children; // sorted for ls
bool isFile;
string content;
```

Use map instead of unordered_map if ls needs lexicographic order.

24. Peeking Iterator

24.1 Problem Summary

Given iterator with:

```
next()
hasNext()
```

Implement:

```
peek()
```

without advancing.

24.2 Key Idea

Cache next value.

24.3 C++ Code

```
class PeekingIterator : public Iterator {
private:
    bool hasPeeked;
    int peekedValue;

public:
    PeekingIterator(const vector<int>& nums) : Iterator(nums) {
        hasPeeked = false;
```

```

    }

    int peek() {
        if (!hasPeeked) {
            peekedValue = Iterator::next();
            hasPeeked = true;
        }

        return peekedValue;
    }

    int next() {
        if (hasPeeked) {
            hasPeeked = false;
            return peekedValue;
        }

        return Iterator::next();
    }

    bool hasNext() const {
        return hasPeeked || Iterator::hasNext();
    }
};

```

25. Common Design Patterns

25.1 Cache Pattern

Use:

hash map + linked list

For LRU.

Use:

hash maps + frequency lists

For LFU.

25.2 Stream Window Pattern

Use:

queue + running sum/count

For recent events.

25.3 Versioned Data Pattern

Use:

index/key -> sorted vector of changes

For Snapshot Array and TimeMap.

25.4 Random O(1) Pattern

Use:

vector + index map

25.5 Dynamic Median Pattern

Use:

two heaps

25.6 Path Tree Pattern

Use:

trie of path components

26. Common Design Edge Cases

26.1 Capacity Zero

Important for:

LRU Cache

LFU Cache

26.2 Updating Existing Key

Should usually update value and recency/frequency.

26.3 Removing Last Element

Important for RandomizedSet.

26.4 Duplicate Values

Need set of indices, not one index.

26.5 Empty Data Structure

What should getMin, getMax, getRandom, or feed return?

Usually problem defines this.

26.6 Stale Heap Entries

If using heap with lazy deletion, top may be outdated.

Always validate heap top before using.

27. Common Design Mistakes

Mistake 1 — One Data Structure Not Enough

LRU cannot be solved by hashmap alone.

Need recency order.

Mistake 2 — Forgetting to Synchronize Structures

If vector changes, update map.

If list node moves, update iterator.

If frequency changes, update bucket.

Mistake 3 — Not Handling Existing Key

In caches, `put(existingKey)` should not create duplicate node.

Mistake 4 — Bad Eviction Tie-Breaker

LFU requires LRU among same frequency.

Mistake 5 — Not Removing Empty Buckets

AllOne and LFU need bucket cleanup.

Mistake 6 — Iterator Invalidations

Be careful when erasing from list/map/vector.

list iterators remain valid except erased element.

vector iterators can invalidate after `resize/push`.

28. Interview Explanation Templates

28.1 LRU Cache

I combine a hash map and a doubly linked list. The hash map gives $O(1)$ access to nodes, and the linked list maintains recency order. On every get or put, I move the node to the front. Eviction removes from the tail.

28.2 LFU Cache

I track each key's value and frequency. For each frequency, I maintain a recency-ordered list of keys. `minFreq` tells me which frequency bucket to evict from, and the back of that list is the least recently used key among the least frequent keys.

28.3 RandomizedSet

I store values in a vector for $O(1)$ random access and maintain a map from value to index. To delete in $O(1)$, I swap the target with the last element and pop.

28.4 Median Finder

I maintain two heaps: a max heap for the lower half and a min heap for the upper half. I rebalance so their sizes differ by at most one.

28.5 TimeMap / SnapshotArray

I store versioned values as sorted pairs of timestamp or snapshot ID and value. Query uses binary search for the largest version not exceeding the requested one.

29. Complexity Patterns

Min Stack	stack of pairs	$O(1)$
Max Stack	list + map	$O(\log n)$ for max operations
LRU Cache	hashmap + DLL	$O(1)$
LFU Cache	hashmaps + freq lists	$O(1)$ avg

Design	Main Structures	Operation Complexity
RandomizedSet	vector + hashmap	$O(1)$ avg
RandomizedCollection	vector + map to sets	$O(1)$ avg
AllOne	bucket list + hashmap	$O(1)$ avg
MedianFinder	two heaps	add $O(\log n)$, median $O(1)$
TimeMap	hashmap + sorted vectors	set $O(1)$, get $O(\log n)$
SnapshotArray	vector of histories	set $O(1)$, get $O(\log k)$
RecentCounter	queue	amortized $O(1)$
HitCounter	queue or buckets	$O(1)$ or $O(300)$
BrowserHistory	vector + index	$O(1)$ mostly

30. Expanded Practice Set

30.1 Must-Do Medium

37. Min Stack
38. LRU Cache
39. Insert Delete GetRandom $O(1)$
40. Find Median from Data Stream
41. Time Based Key-Value Store
42. Snapshot Array
43. Moving Average from Data Stream
44. Recent Counter
45. Design Browser History

30.2 Strong Interview Practice

46. LFU Cache
47. All $O(1)$ Data Structure
48. Max Stack
49. RandomizedCollection
50. Design Twitter
51. Design Snake Game
52. Design File System
53. Design In-Memory File System
54. Hit Counter

30.3 Hard / Stretch

55. Design Search Autocomplete System
56. Design Excel Sum Formula
57. Design Log Storage System
58. Design Underground System

- 59. Design Movie Renting System
 - 60. Design Skiplist
 - 61. Consistent Hashing preview
-

31. Suggested Study Order

Step 1 — Simple State Designs

Solve:

- 62. Min Stack
- 63. Moving Average
- 64. Recent Counter
- 65. Browser History

Goal:

Understand class state and API sequences.

Step 2 — Hash Map + Vector

Solve:

- 66. RandomizedSet
- 67. RandomizedCollection

Goal:

Learn $O(1)$ delete with swap.

Step 3 — Hash Map + Linked List

Solve:

- 68. LRU Cache
- 69. LFU Cache

Goal:

Master cache eviction patterns.

Step 4 — Heaps and Ordered Structures

Solve:

- 70. MedianFinder
- 71. Max Stack

Goal:

Choose heap/set/list based on operations.

Step 5 — Versioned / Time Designs

Solve:

- 72. TimeMap
- 73. SnapshotArray
- 74. HitCounter

Goal:

Use sorted history and windows.

Step 6 — Complex Simulation

Solve:

- 75. Twitter
- 76. Snake Game
- 77. File System

Goal:

Combine multiple concepts cleanly.

32. Pattern Decision Table

stack + getMin	stack of pairs
stack + popMax	list + map of iterators
cache by recency	hashmap + doubly linked list
cache by frequency	hashmap + frequency lists
random pick $O(1)$	vector
insert/delete/random $O(1)$	vector + index map
duplicates + random	vector + value to index-set
dynamic median	two heaps
recent events	queue
fixed time window high volume	circular buckets
timestamp query	sorted vector + binary search
snapshot query	version history + binary search
path operations	trie/tree of components
min/max key by count	bucketed linked list

33. Mini Problem 1 — First Unique Number

Need support:

```
showFirstUnique()
add(value)
```

Use:

```
queue for order
count map
```

Lazy removal from front.

```
class FirstUnique {
private:
    queue<int> q;
    unordered_map<int, int> count;
```

```

public:
    FirstUnique(vector<int>& nums) {
        for (int x : nums) {
            add(x);
        }
    }

    int showFirstUnique() {
        while (!q.empty() && count[q.front()] > 1) {
            q.pop();
        }

        if (q.empty()) {
            return -1;
        }

        return q.front();
    }

    void add(int value) {
        count[value]++;

        if (count[value] == 1) {
            q.push(value);
        }
    }
};

```

34. Mini Problem 2 — Logger Rate Limiter

Print message only if not printed in last 10 seconds.

```

class Logger {
private:
    unordered_map<string, int> lastTime;

public:
    bool shouldPrintMessage(int timestamp, string message) {
        if (!lastTime.count(message) ||
            timestamp - lastTime[message] >= 10) {

```

```
        lastTime[message] = timestamp;
        return true;
    }

    return false;
}
};
```

35. Mini Problem 3 — Parking System

Simple count-based design.

```
class ParkingSystem {
private:
    vector<int> spaces;

public:
    ParkingSystem(int big, int medium, int small) {
        spaces = {0, big, medium, small};
    }

    bool addCar(int carType) {
        if (spaces[carType] == 0) {
            return false;
        }

        spaces[carType]--;
        return true;
    }
};
```

36. Mini Problem 4 — Underground System

Track check-ins and average travel time.

```
class UndergroundSystem {
private:
    unordered_map<int, pair<string, int>> checkIn;
    unordered_map<string, pair<long long, int>> stats;
```

```

string key(string start, string end) {
    return start + "#" + end;
}

public:
void checkIn(int id, string stationName, int t) {
    checkIn[id] = {stationName, t};
}

void checkOut(int id, string stationName, int t) {
    auto [start, startTime] = checkIn[id];
    checkIn.erase(id);

    string route = key(start, stationName);

    stats[route].first += t - startTime;
    stats[route].second++;
}

double getAverageTime(string startStation, string endStation) {
    string route = key(startStation, endStation);

    auto [total, count] = stats[route];

    return (double)total / count;
}
};

```

37. Common Edge Case Checklist

Before submitting design code, test operation sequences:

- empty structure operation if allowed
- capacity 0
- capacity 1
- insert duplicate
- remove missing value
- remove existing value
- remove last element
- update existing key
- evict after update
- get updates recency

tie-breaking in LFU
multiple same frequency keys
all buckets empty
timestamp before first value
snapshot before any set
browser visit after back
snake moves into old tail

38. Debugging Design Problems

If design code fails, check:

78. Are all required operations meeting target complexity?
79. Are internal structures synchronized after every update?
80. Are existing keys handled separately from new keys?
81. Is eviction rule exactly correct?
82. Are tie-breakers handled?
83. Are iterators still valid after erase?
84. Are dummy nodes linked correctly?
85. Are stale entries possible?
86. Are histories sorted for binary search?
87. Are edge cases like capacity zero handled?
88. Does operation sequence reveal hidden state bugs?

39. What to Master Before Chapter 55

Before moving to System Design Lite / API Design, make sure you can:

- Explain design problem approach
- Choose data structures from API requirements
- Implement Min Stack
- Implement LRU Cache
- Implement LFU Cache conceptually
- Implement RandomizedSet
- Implement MedianFinder
- Implement TimeMap
- Implement SnapshotArray
- Understand AllOne bucket design
- Implement queue-based stream counters
- Simulate Snake Game correctly
- Handle design edge cases across operation sequences

40. Final Cheat Sheet

LRU

hashmap: key -> node
DLL order: most recent near head
get/put existing -> move to head
evict -> remove tail.prev

LFU

key -> value
key -> freq
freq -> recency list of keys
key -> iterator
minFreq

RandomizedSet

vector values
map value -> index
remove by swap with last

MedianFinder

low max heap
high min heap
rebalance sizes

TimeMap

key -> vector of {timestamp, value}
get uses binary search <= timestamp

SnapshotArray

index -> vector of {snapId, value}
set overwrites current snap entry
get uses binary search

Recent Window

queue timestamps
pop timestamps outside window

AllOne

bucket list ordered by count
key -> bucket
inc/dec move between adjacent buckets

41. Core Takeaway

Design Data Structure problems are about matching API requirements to internal structures.

The key question is:

What must be fast?

Then choose structures that make those operations fast.

Most strong designs combine:

- hash map for lookup
- linked list for order
- vector for random access
- heap for priority
- queue for time windows
- sorted vector for version search
- trie/tree for path hierarchy

The hardest part is not choosing the structures.

The hardest part is keeping them synchronized after every operation.

Always test sequences, not just single calls.
